RDF 1.2 Concepts and Abstract Data Model

W3C Working Draft 28 January 2026



▼ **More details about this document**

This version:

https://www.w3.org/TR/2026/WD-rdf12-concepts-20260128/

Latest published version:

https://www.w3.org/TR/rdf12-concepts/

Latest editor's draft:

https://w3c.github.io/rdf-concepts/spec/

History:

https://www.w3.org/standards/history/rdf12-concepts/

Commit history

Test suite:

https://w3c.github.io/rdf-tests/rdf/rdf12/

Implementation report:

https://w3c.github.io/rdf-tests/rdf/rdf12/reports/

Latest Recommendation:

https://www.w3.org/TR/rdf11-concepts

Editors:

Gregg Kellogg (until 2025-09-06), *in memoriam*

Olaf Hartig

Pierre-Antoine Champin

Andy Seaborne

Former editors:

Richard Cyganiak (RDF 1.1)

David Wood (RDF 1.1, Chair)

Markus Lanthaler (RDF 1.1)

Graham Klyne (RDF 1.0)

Jeremy J. Carroll (RDF 1.0)

Brian McBride (RDF 1.0, Chair)

Feedback:

GitHub w3c/rdf-concepts (pull requests, new issue, open issues)

public-rdf-star-wg@w3.org with subject line `[rdf12-concepts] … message topic …` (archives)

Copyright © 2004-2026 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

The Resource Description Framework (RDF) is a framework for representing information on the Web. This document defines an abstract data model which serves to link all RDF-based languages and specifications. The abstract data model has two key data structures:

- RDF graphs are sets of subject-predicate-object triples, where the elements may be IRIs, blank nodes, datatyped literals, or triple terms. They are used to express descriptions of resources.
- RDF datasets are used to organize collections of RDF graphs, and consist of a default graph and zero or more named graphs.

Compared to RDF 1.1, RDF 1.2 introduces the ability to use an RDF triple as a triple term, in the object position of another triple. RDF 1.2 also introduces directional language-tagged strings, which contain a base direction component that allows the initial text direction to be specified for presentation by a user agent. Finally, to ease the transition from RDF 1.1 to RDF 1.2, this specification introduces a mechanism for explicitly conveying the version of RDF that is used by a given piece of data.

This specification introduces key concepts and terminology for RDF 1.2, and subsequently discusses datatyping and the handling of fragment identifiers in IRIs within RDF graphs.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document is part of the RDF 1.2 document suite. It is the central RDF 1.2 specification and defines the core RDF concepts. Test suites and implementation reports of a number of RDF 1.2 specifications that build on this document are available from the rdf-tests repository under rdf-tests/rdf/rdf12 as referenced in each of the specifications.

RDF 1.2 Concepts is an update to [*RDF11-CONCEPTS*], which was itself, an update to [*RDF-CONCEPTS-20040210*]. The tests described in [*RDF11-TESTCASES*] are valid for RDF 1.2.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this upcoming Recommendation may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

Table of Contents

Abstract

Status of This Document
Set of Documents

1. Introduction
1.1 Graph-based Abstract Data Model
1.2 Resources and Statements
1.3 The Referent of an IRI
1.4 RDF Vocabularies and Namespace IRIs
1.5 Triple Terms and Reification
1.6 RDF and Change over Time
1.7 Working with Multiple RDF Graphs
1.8 Equivalence, Entailment and Inconsistency
1.9 RDF Documents and Syntaxes
1.10 RDF Version Announcement

2. Conformance
2.1 Version Labels
2.2 Strings in RDF

3. RDF Graphs
3.1 Triples
3.2 RDF Terms
3.3 IRIs
3.4 Literals
3.5 Blank Nodes
3.6 Triple Terms
3.7 Graph Comparison

4. RDF Datasets
4.1 RDF Dataset Comparison
4.2 Content Negotiation of RDF Datasets
4.3 Dataset as a Set of Quads

5. Datatypes
5.1 The XML Schema Built-in Datatypes
5.2 Datatype IRIs

6. **Fragment Identifiers**

7. **Generalizations of RDF Triples, Graphs, and Datasets**
7.1 Symmetric RDF
7.2 Generalized RDF

A. **Additional Datatypes**
A.1 The `rdf:HTML` Datatype
A.2 The `rdf:XMLLiteral` Datatype
A.3 The `rdf:JSON` Datatype

B. **Replacing Blank Nodes with IRIs**

C. **Privacy Considerations**

D. **Security Considerations**

E. **Internationalization Considerations**

F. **IRI Grammar**

G. **Acknowledgments**
G.1 Acknowledgments for RDF 1.0
G.2 Acknowledgments for RDF 1.1
G.3 Acknowledgments for RDF 1.2

H. **Changes between RDF 1.1 and RDF 1.2**

I. **Index**
I.1 Terms defined by this specification
I.2 Terms defined by reference

J. **References**
J.1 Normative references
J.2 Informative references

§ 1. Introduction

This section is non-normative.

The *Resource Description Framework* (RDF) is a framework for representing information on the Web.

This document defines an abstract data model which serves to link all RDF-based languages and specifications, including the following:

- the formal model-theoretic semantics for RDF [*RDF12-SEMANTICS*]
- serialization syntaxes for storing and exchanging RDF such as *RDF 1.2 N-Triples* [*RDF12-N-TRIPLES*], *RDF 1.2 Turtle* [*RDF12-TURTLE*], and *JSON-LD 1.1* [*JSON-LD11*]
- the *SPARQL 1.2 Query Language* [*SPARQL12-QUERY*]
- the *RDF 1.2 Schema* [*RDF12-SCHEMA*]

§ 1.1 Graph-based Abstract Data Model

The core structure of the abstract data model is a set of triples, each consisting of a subject, a predicate and an object. A set of such triples is called an RDF graph. An RDF graph can be visualized as a node and directed-arc diagram, in which each triple is represented as a node-arc-node link.

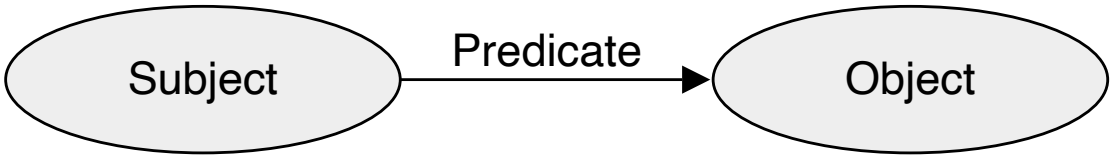


Figure 1 An RDF graph with two nodes (Subject and Object) and an arc (Predicate) connecting them.

There are four kinds of nodes that can be in an RDF graph: IRIs, literals, blank nodes, and triple terms.

From this definition, it follows that when one term appears in multiple triples, these are simply multiple occurrences of that same term. For example, in a graph containing two triples (here expressed in common set and tuple notation, and using abstract names as distinct terms):

```
{ (R1, P1, R3),
  (R2, P2, R3) }
```

the term R3 is the same single term used twice, and there are five terms in total. This is more readily shown in two-dimensional graph diagrams, where one single node can simply be connected from or to multiple other nodes using labelled arcs.

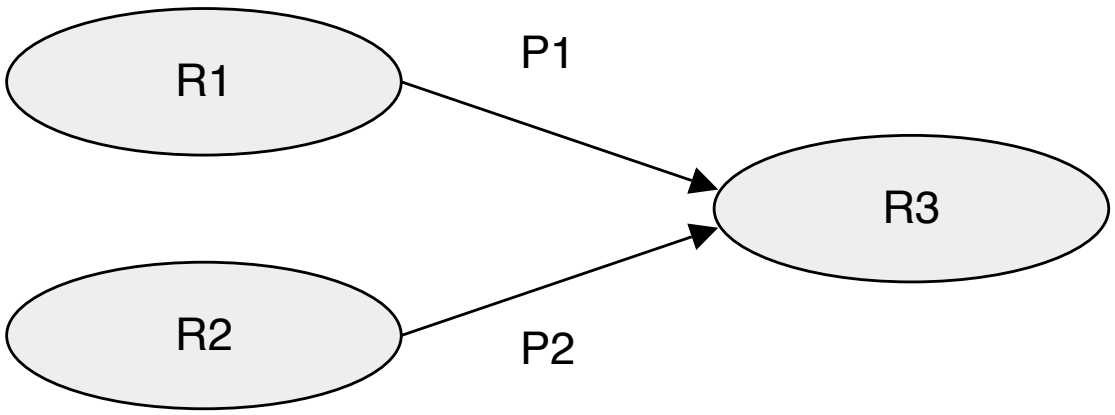


Figure 2 An RDF graph with three nodes (R1, R2, and R3) and two arcs (P1 and P2), the arcs respectively connecting R1 and R2 to R3.

This abstract data model can be encoded in different ways while preserving the same stucture, as described in 1.9 RDF Documents and Syntaxes.

> NOTE
> The graph structure of the abstract data model is not a conceptual model. It is a symbolic, structural basis for such modelling, enabling definition and use of RDF vocabularies, and semantic ***interpretation*** [*RDF12-SEMANTICS*]. Practical examples are given in the *RDF 1.2 Primer* [*RDF12-PRIMER*].

§ 1.2 Resources and Statements

Any IRI or literal ***denotes*** something in the world (the "universe of discourse"). These things are called ***resources***. Anything can be a resource, including physical things, documents, abstract concepts, numbers and strings; the term is synonymous with "entity" as it is used in *RDF 1.2 Semantics* [*RDF12-SEMANTICS*]. The resource denoted by an IRI is called its referent, and the resource denoted by a literal is called its literal value. Literals have datatypes that define the range of possible values, such as strings, numbers, and dates. Special kinds of literals — language-tagged strings and directional language-tagged strings — respectively denote plain-text strings in a natural language and plain-text strings in a natural language including an initial text direction.

Asserting an RDF triple says that *some relationship, indicated by the predicate, holds between the resources denoted by the subject and object* (as explained below, not all triples are asserted). This statement corresponding to an RDF triple is known as an ***RDF statement***. The predicate itself is an IRI and denotes a ***property***, that is, a resource that can be thought of as a binary relation. (Relations that involve more than two entities can only be indirectly expressed in RDF [*SWBP-N-ARYRELATIONS*].)

Unlike IRIs and literals, blank nodes do not identify specific resources. Statements involving blank nodes say that something with the given relationships exists, without explicitly naming it.

§ 1.3 The Referent of an IRI

The resource denoted by an IRI is also called its ***referent***. For some IRIs with particular meanings, such as those identifying XSD datatypes, the referent is fixed by this specification. For all other IRIs, what exactly is denoted by any given IRI is not defined by this specification. Other specifications may fix IRI referents, or apply other constraints on what may be the referent of any IRI.

Guidelines for determining the referent of an IRI are provided in other documents, like *Architecture of the World Wide Web, Volume One* [*WEBARCH*] and *Cool URIs for the Semantic Web* [*COOLURIS*]. A very brief, informal, and partial account follows:

- By design, IRIs have global scope. Thus, two different appearances of an IRI denote the same resource. Violating this principle constitutes an IRI collision [*WEBARCH*].
- By social convention, the IRI owner [*WEBARCH*] gets to say what the intended (or usual) referent of an IRI is. Applications and users need not abide by this intended denotation, but there may be a loss of interoperability with other applications and users if they do not do so.
- The IRI owner can establish the intended referent by means of a specification or other document that explains what is denoted. For example, the *The Organization Ontology* [*VOCAB-ORG*] specifies the intended referents of various IRIs that start with `http://www.w3.org/ns/org#`.
- A good way of communicating the intended referent is to set up the IRI so that it dereferences [*WEBARCH*] to such a document.
- Such a document can, in fact, be an RDF document that describes the denoted resource by means of RDF statements.

Perhaps the most important characteristic of IRIs in web architecture is that they can be dereferenced, and hence serve as starting points for interactions with a remote server. This specification is not concerned with such interactions. It does not define an interaction model. It only treats IRIs as globally unique identifiers in a graph data model that describes resources. However, those interactions are critical to the concept of *Linked Data Design Issues*, [*LINKED-DATA*], which uses the RDF abstract syntax and concrete RDF syntaxes, the latter also referred to as serialization formats.

§ 1.4 RDF Vocabularies and Namespace IRIs

An ***RDF vocabulary*** is a collection of IRIs intended for use in RDF graphs. For example, the IRIs documented in [*RDF12-SCHEMA*] are the RDF Schema vocabulary. RDF Schema can itself be used to define and document additional RDF vocabularies. Some such vocabularies are mentioned in the *RDF 1.2 Primer* [*RDF12-PRIMER*].

The IRIs in an RDF vocabulary often begin with a common substring known as a ***namespace IRI***. Some namespace IRIs are associated by convention with a short name known as a ***namespace prefix***.

Some namespace prefixes and IRIs used in this specification

Some namespace prefixes and IRIs used in this specification

Namespace prefix	Namespace IRI	RDF vocabulary
rdf	`http://www.w3.org/1999/02/22-rdf-syntax-ns#`	The RDF built-in vocabulary [*RDF12-SCHEMA*]
rdfs	`http://www.w3.org/2000/01/rdf-schema#`	The RDF Schema vocabulary [*RDF12-SCHEMA*]
xsd	`http://www.w3.org/2001/XMLSchema#`	The RDF-compatible XSD types

In some serialization formats, it is common to associate some namespace IRIs with arbitrary namespace prefixes, and to improve readability by abbreviating IRIs that start with one of those namespace IRIs by using the corresponding namespace prefix. For example, based on the prefix mapping in the table above, the IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#XMLLiteral` would be abbreviated as `rdf:XMLLiteral`. Note however that such abbreviations are *not* meant to be processed directly as IRIs, and are not to be used in syntactic contexts where IRIs are expected. Note also that namespace IRIs and namespace prefixes are not a formal part of the RDF abstract data model. They are merely a syntactic convenience for abbreviating IRIs; for processing, the actual IRIs are reconstructed by replacing each namespace prefix with the corresponding namespace IRI.

The term "***namespace***" on its own does not have a well-defined meaning in the context of RDF, but is sometimes informally used to mean "namespace IRI" or "RDF vocabulary".

§ 1.5 Triple Terms and Reification

A triple term is an RDF triple used as an RDF term within another triple. Being an RDF triple, it denotes a proposition.

A ***reifying triple*** is a triple where the predicate is `rdf:reifies` and the object is a triple term. The subject of that triple is called a ***reifier***, and it can be the subject or object of other triples.

A reifier may denote a variety of things that are related to the triple term's proposition, such as a statement or belief that the proposition holds. It is expected that the reifiers (rather than the triple terms) will be used in further statements. This section briefly describes this common usage. For more examples, refer to the *RDF 1.2 Primer* [*RDF12-PRIMER*].

For example, the following diagram represents a reifying triple of a triple term, together with a triple that includes the reifier as the subject. The latter describes the reifier as a claim, made by `:Bob`, that the proposition denoted by the reified triple term holds. In other words, `:Bob` claims that `:Alice`'s family name is "Liddell".

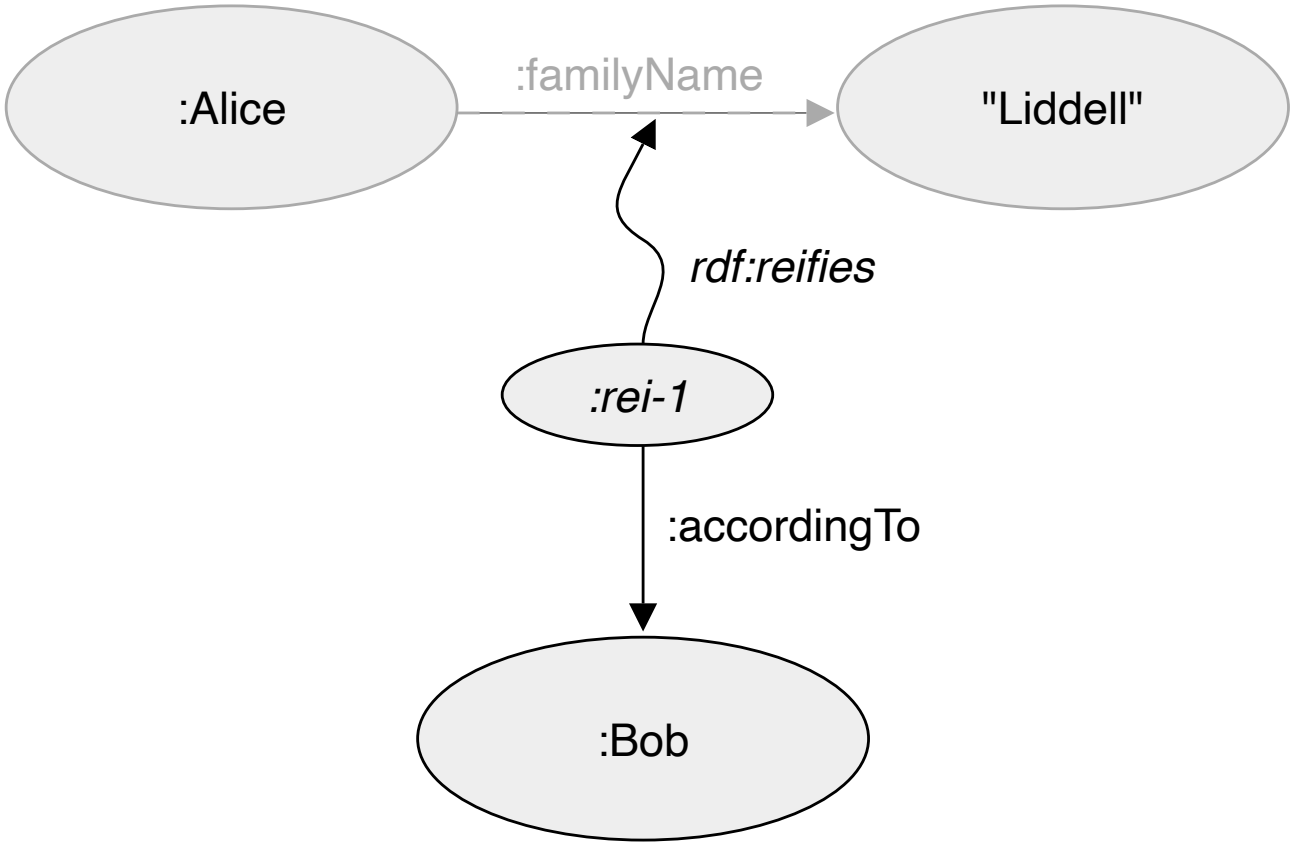


Figure 3 An RDF graph containing a reifying triple that references a triple term (which is unasserted, depicted using a grey, dashed arc) from a reifier; and a triple describing this reifier.

In this example, the proposition denoted by the triple term (i.e., the proposition that `:Alice`'s family name is `"Liddell"`) is not claimed to be true. That would only be the case if the triple used as a triple term was also an asserted triple in the RDF graph. By using non-asserted triple

terms, as in the figure, one can make statements about unasserted statements; for example, if one is unsure whether `:Alice`'s family name is actually `"Liddell"`.

Here is a variation on the graph shown in Figure 3. This represents a graph where an asserted triple corresponds to the triple term object of a reifying triple. In this case, the subset of triples including the reifier as subject—as illustrated in these examples— is called a ***triple annotation***.

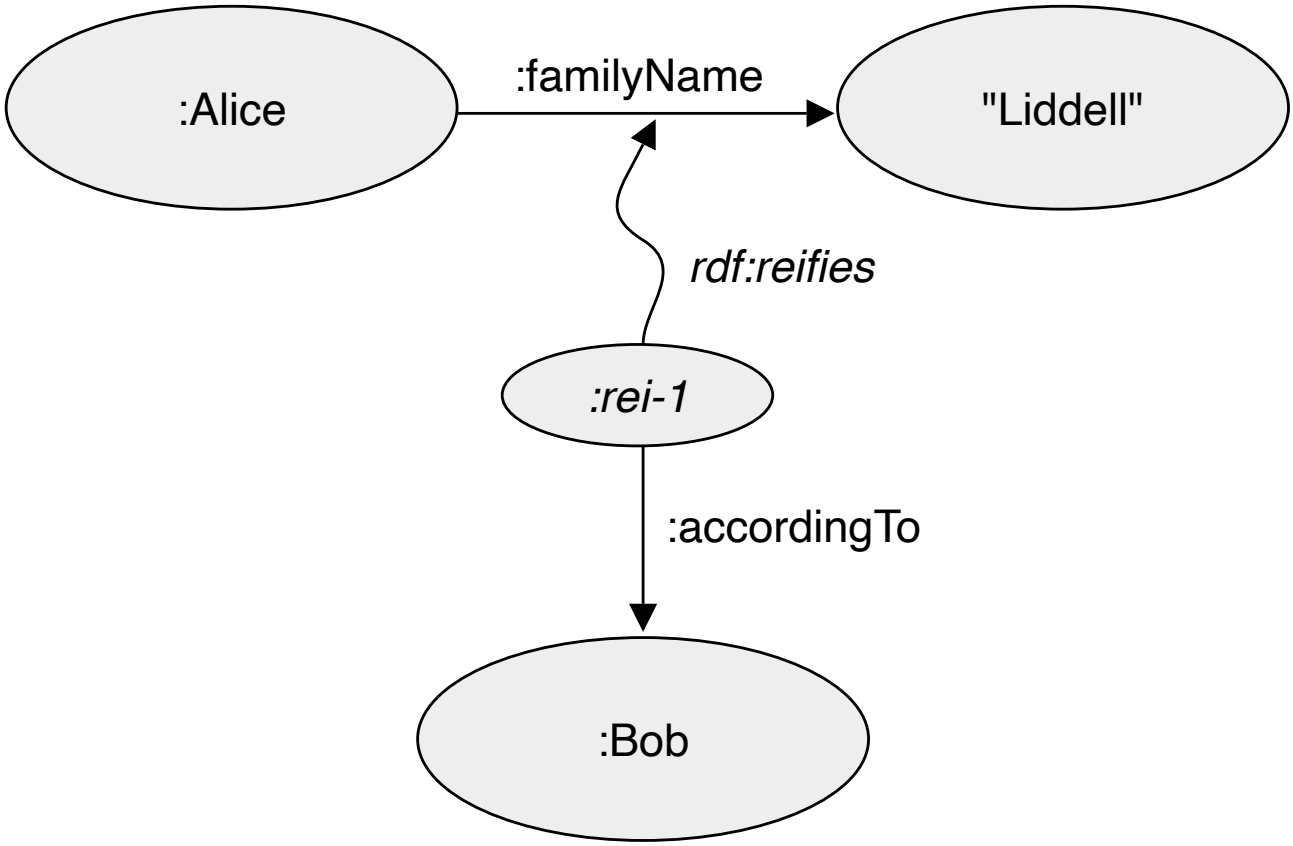


Figure 4 An RDF graph containing a triple annotation, where the triple term of a reifying triple corresponds to an asserted triple. Due to the asserted triple, the diagram represents the proposition as a fact, meaning that the relationship holds.

Concrete syntaxes, such as Turtle [*RDF12-TURTLE*], may have shortcuts for specifying reifying triples and triple annotations more succinctly.

Finally, RDF terms that appear in a triple term have the same denotation as when they appear in an asserted triple in the graph. For example, the term `:Alice` from a triple term and `:Alice` in an asserted triple both denote the same resource. For this reason, we say that triple terms are ***transparent***.

NOTE

As already stated, reifiers are meant to serve a broad range of use cases: statements or beliefs that a proposition is true, situations in which the proposition is true, events that caused the proposition to become true, etc. Because of this diversity, the meaning of the `rdf:reifies` property is deliberately generic.

There can be multiple, distinct reifiers related to the same abstract proposition, such as statements with different sources, or situations with different characteristics. One reifier may also be used to reify multiple, distinct propositions, expressing for example the fact that the same situation could stem from different propositions.

Since a proposition that is reified does not have to hold, it is possible to make statements about any kind of statement, including an unasserted statement that contradicts another statement, whether asserted or not.

§ 1.6 RDF and Change over Time

The RDF abstract data model is *atemporal*: RDF graphs are static snapshots of information.

However, RDF graphs can express information about events and about temporal aspects of other entities, given appropriate vocabulary terms.

Since RDF graphs are defined as mathematical sets, adding or removing triples from an RDF graph yields a different RDF graph.

We informally use the term ***RDF source*** to refer to a persistent yet mutable source or container of RDF graphs. An RDF source is a resource that may be said to have a state that can change over time. A snapshot of the state can be expressed as an RDF graph. For example, any web document that has an RDF-bearing representation may be considered an RDF source. Like all resources, RDF sources may be named with IRIs and therefore described in other RDF graphs.

Intuitively speaking, changes in the universe of discourse can be reflected in the following ways:

- An IRI, once minted, should never change its intended referent. (See URI persistence [*WEBARCH*].)
- Literals, by design, are constants and never change their value.
- A relationship that holds between two resources at one time may not hold at another time.
- RDF sources may change their state over time. That is, they may provide different RDF graphs at different times.
- Some RDF sources may, however, be immutable snapshots of another RDF source, archiving its state at some point in time.

§ 1.7 Working with Multiple RDF Graphs

As RDF graphs are sets of triples, they can be combined easily, supporting the use of data from multiple sources. Nevertheless, it is sometimes desirable to work with multiple RDF graphs while keeping their contents separate. RDF datasets support this requirement.

An RDF dataset is a collection of RDF graphs. All but one of these graphs have an associated IRI or blank node. They are called named graphs, and the IRI or blank node is called the graph name. The remaining graph does not have an associated IRI, and is called the default graph of the RDF dataset.

There are many possible uses for RDF datasets. One such use is to hold snapshots of multiple RDF sources.

§ 1.8 Equivalence, Entailment and Inconsistency

An RDF triple denotes a ***proposition*** — a simple logical expression, describing a relationship between two entities. An asserted triple is a claim that the corresponding proposition is true. An RDF graph is the conjunction (logical *AND*) of all the claims made by its asserted triples. The precise details of this meaning of RDF triples and RDF graphs are the subject of *RDF 1.2 Semantics* [*RDF12-SEMANTICS*], which yields the following relationships between RDF graphs:

Entailment

An RDF graph *A* entails another RDF graph *B* if every possible arrangement of the world that makes *A* true also makes *B* true. When *A* entails *B*, if the truth of *A* is presumed or demonstrated then the truth of *B* is established.

Equivalence

Two RDF graphs *A* and *B* are equivalent if they make the same claim about the world. *A* is equivalent to *B* if and only if *A* entails *B* and *B* entails *A*.

Inconsistency

An RDF graph is inconsistent if it contains an internal contradiction. There is no possible arrangement of the world that would make the expression true.

An ***entailment regime*** [*RDF12-SEMANTICS*] is a specification that defines precise conditions that make these relationships hold. RDF itself recognizes only some basic cases of entailment, equivalence and inconsistency. Other specifications, such as *RDF 1.2 Schema* [*RDF12-SCHEMA*] and OWL 2 [*OWL2-OVERVIEW*], add more powerful entailment regimes, as do some domain-specific vocabularies.

This specification does not constrain how implementations use the logical relationships defined by entailment regimes. Implementations may or may not detect inconsistencies, and may make all, some or no entailed information available to users.

§ 1.9 RDF Documents and Syntaxes

An ***RDF document*** is a document that encodes an RDF graph or RDF dataset in a ***concrete RDF syntax***, such as N-Triples [*RDF12-N-TRIPLES*], Turtle [*RDF12-TURTLE*], RDFa [*RDFA-CORE*], JSON-LD [*JSON-LD11*], or TriG [*RDF12-TRIG*]. RDF documents enable the exchange of RDF graphs and RDF datasets between systems.

A concrete RDF syntax may offer many different ways to encode the same RDF graph or RDF dataset, for example through the use of namespace prefixes, IRI references, blank node identifiers, and different ordering of triples. While these aspects can have great effect on the convenience of working with the RDF document, they are not significant for its meaning.

The basis for concrete RDF syntaxes is the structure of the abstract data model, called the ***abstract syntax***; summarized in the following two tables.

The abstract syntax of RDF graphs

The abstract syntax of RDF graphs

Production	Defined as
RDF graph	a *set* of *zero or more* triples
triple	a *3-tuple* of a subject, a predicate, and an object
subject	*either* an IRI *or* a Blank node
predicate	an IRI
object	*either* an IRI *or* a Blank node *or* a Literal *or* a triple

The abstract syntax of RDF datasets

Production	Defined as
RDF dataset	a *pair* of a default graph and a *set* of *zero or more* named graphs
default graph	an RDF graph
named graph	a *pair* of a graph name and an RDF graph
graph name	*either* an IRI *or* a Blank node

§ 1.10 RDF Version Announcement

To allow RDF parsers to error or warn about unsupported RDF versions as early as possible, RDF serialization formats are expected to allow a version to be specified, via either a media-type parameter, a version announcement in a format-specific syntax, or both.

When the version is indicated both in a media-type parameter and in syntax, they are expected to be the same. If they differ, parsers use the version from the media-type parameter and might emit a warning about the mismatch.

To retain compability that is as broad as possible with older parsers, only RDF documents that make use of RDF 1.2-specific functionality are encouraged to announce their version (i.e., for RDF documents that do not make use of RDF 1.2-specific functionality it is discouraged to announce a version).

To announce the version in the HTTP responses using the `Content-Type` header, the server is expected to use the `version` parameter, as illustrated in the following example response.

```
HTTP/1.1 200 OK
Content-Type: text/turtle; version=1.2
Location: http://example.com/document.ttl
```

Servers are also expected to announce the version in-line, when the format supports in-line version announcement (such as *RDF 1.2 Turtle* [*RDF12-TURTLE*]).

When requesting an RDF document from an HTTP server, a client can use the `version` parameter during content negotiation [*WEBARCH*], by including it in the `Accept` request header, as illustrated in the following example request.

```
GET /document.ttl HTTP/1.1
Host: example.com
Accept: text/turtle; version=1.2
```

Section 2.1 Version Labels defines version labels to be used with the `version` parameter and in concrete RDF syntax.

> NOTE
>
> As HTTP content negotiation is advisory, clients receiving a document should be prepared to properly handle a document of the requested media type but potentially having a `version` other than what was requested. Clients may consider down-grading the content to an appropriate version themselves as discussed in 2.1.1 Server Considerations.

§ 2. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, *RECOMMENDED*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

This specification, *RDF 1.2 Concepts and Abstract Data Model*, defines an abstract data model and related vocabulary for use in other specifications, such as concrete RDF syntaxes, API specifications, and query languages. Implementations cannot directly conform to *RDF 1.2 Concepts and Abstract Data Model*, but can conform to such other specifications that normatively reference terms defined here.

This specification establishes two conformance levels:

- ***Full conformance*** supports graphs and datasets with triples that contain triple terms.
- ***Basic conformance*** only supports graphs and/or datasets with triples that only contain basic RDF terms — i.e., they do not contain triple terms.

§ 2.1 Version Labels

A ***version label*** is a string that identifies the syntax and semantics conformance for the RDF data.

Version Labels

Version Label	Syntax	Semantics
"1.2"	RDF 1.2 syntax	RDF 1.2 Semantics
"1.2-basic"	RDF 1.2 syntax without triple terms	RDF 1.2 Semantics
"1.1"	RDF 1.1 syntax except for use of a version directive	RDF 1.1 Semantics

Data conforming to version "1.1" is valid as data with version "1.2-basic", and data conforming to version "1.2-basic" is valid as data with version "1.2". Data conforming to version "1.1" has the same semantics under *RDF 1.1 Semantics* and under *RDF 1.2 Semantics*.

See *RDF 1.2 Interoperability* for details of encoding "1.2" as "1.2-basic".

For serializations supporting in-line version announcement, the version announcement *SHOULD* be made early in the document and certainly before serializing any feature depending on that version.

§ 2.1.1 Server Considerations

This section is non-normative.

When serializing a graph or dataset which uses features incompatible with a requested version, servers can consider different alternatives:

1. Eliminate triple terms by using an algorithm such as *basic encoding* as defined in *RDF 1.2 Interoperability* (downgrading from version "1.2" to "1.2-basic").
2. Replace directional language-tagged strings by literals with a datatype IRI that uses the `i18n` namespace, as defined in [*JSON-LD11*] (downgrading from version "1.2" or "1.2-basic" to "1.1").
3. Return `406 "Not Acceptable"`.
4. Ignore the requested version and return a native representation.

> NOTE
>
> The suggestion here is to follow the Robustness principle (also known as Postel's Law): servers should be conservative in what they send, be liberal in what they accept.

§ 2.1.2 Client Considerations

This section is non-normative.

If a document has no stated `version` (either by HTTP header or internal), systems can assume the version is `"1.2"`, which continues to support all prior RDF versions.

When parsing a document with conflicting versions, or when parsing a document using unsupported versions, a parser may do any of the following:

- Ignore the version directive.
- Raise an error and abort.
- Issue a warning and ignore the triple.
- Parse the structure and only emit triples consistent with the declared version.

§ 2.2 Strings in RDF

RDF uses Unicode [*Unicode*] as the fundamental representation for string values. Within this and related specifications, the term ***RDF string***, or simply string, is used to describe an ordered sequence of zero or more Unicode code points which are Unicode scalar values. Unicode scalar values do not include the surrogate code points. Note that most concrete RDF syntaxes require the use of the UTF-8 character encoding [*RFC3629*], and use the `\u0000` or `\U00000000` forms to express certain non-character values.

A string is identical to another string if it consists of the same sequence of code points. An implementation *MAY* determine string equality by comparing the code units of two strings that use the same Unicode character encoding (UTF-8 or UTF-16) without decoding the string into a Unicode code point sequence.

§ 3. RDF Graphs

An ***RDF graph*** is a set of RDF triples.

An RDF triple that is an element of an RDF graph is also said to be ***asserted*** in that RDF graph.

§ 3.1 Triples

An ***RDF triple*** (often simply called "triple") is a 3-tuple that is defined inductively as follows:

- If *s* is an IRI or a blank node, *p* is an IRI, and *o* is an IRI, a blank node, or a literal, then (*s*, *p*, *o*) is an RDF triple.
- If *s* is an IRI or a blank node, *p* is an IRI, and *o* is an RDF triple, then (*s*, *p*, *o*) is an RDF triple.

The three components (*s*, *p*, *o*) of an RDF triple are respectively called the ***subject***, ***predicate*** and ***object*** of the triple.

> NOTE
>
> The definition of triple is recursive. That is, a triple can itself have an object component which is another triple. However, by this definition, cycles of triples cannot be created.

The set of ***nodes*** of an RDF graph is the set of subjects and objects of the asserted triples of the graph. It is possible for a predicate IRI to also occur as a node in the same graph.

Triple equality: Two triples (*s*, *p*, *o*) and (*s*', *p*', *o*') are equal (the same RDF triple) if and only if all of the following three conditions hold.

- *s* and *s*' are equal.
- *p* and *p*' are equal.
- *o* and *o*' are equal.

§ 3.2 RDF Terms

IRIs, literals, blank nodes, and triple terms are collectively known as ***RDF terms***.

IRIs, literals and blank nodes are said to be ***basic RDF terms***.

RDF term equality: Two RDF terms *t* and *t*' are equal (the same RDF term) if and only if one of the following four conditions holds:

- *t* and *t*' are IRIs that are equal (per IRI equality).
- *t* and *t*' are literals that are equal (per literal term equality).
- *t* and *t*' are blank nodes that are equal (per blank node equality).

- *t* and *t'* are triple terms that are equal (per triple equality).

> NOTE
> From the above definition of RDF term equality, it follows that terms of different kinds (IRI, literal, blank node, or triple term) are always distinguishable, even if they are otherwise based on the same string. For example, the IRI `http://example.org/` is not equal to a literal whose lexical form is `http://example.org/`.

The set of RDF terms ***appearing*** in an RDF triple *t* is defined inductively as follows:

- The subject, predicate and object of *t* appear in *t*.
- If the object of *t* is an RDF triple *t2*, then any RDF term appearing in *t2* also appears in *t*.

By extension, an RDF term is said to appear in an RDF graph if it appears in an asserted triple of that graph. An RDF triple is said to appear in an RDF graph if it is either an asserted triple of that graph or a triple term appearing in that graph.

An RDF term is said to be ***ground*** if any of the following three conditions holds:

- It is an IRI.
- It is a literal.
- It is a triple term (s, p, o) such that s, p, and o are all ground.

By extension, an RDF triple is said to be ground if its subject, predicate, and object are all ground. An RDF graph is said to be ground if all its asserted triples are ground.

§ 3.3 IRIs

An ***IRI*** (Internationalized Resource Identifier) within an RDF graph is a string that conforms to the syntax defined in RFC 3987 [*RFC3987*].

An IRI in the RDF abstract syntax *MUST* be resolved per [*RFC3986*] and *MUST NOT* be a relative reference. An IRI *MAY* contain a fragment identifier. An IRI *SHOULD* follow rules defined by the IRI scheme.

IRI equality: Two IRIs are equal if and only if they consist of the same sequence of Unicode code points, as in Simple String Comparison in section 5.3.1 of [*RFC3987*]. (This is done in the abstract syntax, so the IRIs are resolved IRIs with no escaping or encoding.) Further normalization *MUST NOT* be performed before this comparison.

NOTE

For convenience, a complete [*ABNF*] grammar from [*RFC3987*] is provided in F. IRI Grammar.

NOTE

URIs and IRIs: IRIs are a generalization of ***URIs*** [*RFC3986*] that permits a wider range of Unicode characters [*UNICODE*]. Every URI and URL is an IRI, but not every IRI is an URI. In RDF, IRIs are used as ***IRI references***, as defined in [*RFC3987*] section 1.3. An IRI reference is common usage of an Internationalized Resource Identifier. An IRI reference refers to either a resolved IRI or relative IRI reference, as described by the *IRI-reference* production in F. IRI Grammar. The abstract syntax uses only fully resolved IRIs. When IRIs are used in operations that are only defined for URIs, they must first be converted according to the mapping defined in section 3.1 of [*RFC3987*]. A notable example is retrieval over the HTTP protocol. The mapping involves UTF-8 encoding of non-ASCII characters, %-encoding of octets not allowed in URIs, and Punycode-encoding of domain names.

URLs: The *URL Standard* is largely compatible with [*RFC3987*] IRIs, but is based on a processing model important for implementation within web browsers and are not described using an [*ABNF*] grammar.

§ 3.3.1 RDF Reference IRIs

This section is non-normative.

This section provides advice to data publishers.

IRIs are used to denote resources, and each IRI should identify the same resource regardless of where that IRI is used. Note that the general syntax for IRIs, defined by [*RFC3987*], can express IRIs which do not meet the requirement of being a global reference. Some URI schemes add additional requirements; for example, the HTTP URI scheme defines `http-URI`, which requires the presence of a non-empty host name, and, as a consequence, the path component will start with `/`. The [*RFC3987*] syntax permits IRIs such as `http:abcd` and `http:///abcd`, but these are invalid because they do not satisfy the HTTP URI scheme definition.

An ***RDF Reference IRI***, sometimes called simply RDF Reference, is an IRI that is suitable for use as a global reference.

Reference Resolution: An RDF Reference IRI is unchanged by reference resolution. The path component starts with a `/` character and does not contain `.` or `..` segments.

Relative IRI references: Some concrete RDF syntaxes permit ***relative IRI references*** (see the `irelative-ref` production in the IRI Grammar) as a convenient shorthand that allows RDF documents to be authored without knowing their final publishing location. Relative IRI references must be resolved against a ***base IRI***. Therefore, the RDF graph serialized in such syntaxes is well-defined only if a base IRI can be established [*RFC3986*].

URI Schemes: Implementations are encouraged to follow the scheme-specific rules of the common schemes, such as the scheme rules for HTTP/HTTPS and the DID syntax. Implementations ignore URI scheme rules for schemes they do not recognize.

IRI normalization: Interoperability problems can be avoided by minting only IRIs that are normalized according to Section 5 of [*RFC3987*].

- Use lowercase characters in scheme names.
- Only use percent-encoding of characters where required by the IRI syntax.
- Omit the HTTP or HTTPS default port; `http://example/` is preferred over `http://example:80/`.
- Use uppercase hexadecimal letters within percent-encoding triplets;"`%3F`" is preferred over "`%3f`".
- An empty path in an HTTP IRI `http://example/` is preferred over having no path `http://example`.
- Normalize IRIs to remove "`/./`" and "`/../`" in the path component of an IRI.

- Use lowercase characters in domain names. Note that, while ASCII characters in domain names are case-insensitive, non-ASCII characters in domain names are case-sensitive [*RFC5890*]. Domains are generally only registered with lowercase letters [*RFC5892*].
- Avoid using the A-label (ASCII, punycode-encoded name]) for Internationalized Domain Names [*RFC5890*] in IRIs.
- Use IRIs in Unicode Normalization Form C [*I18N-Glossary*].

§ 3.4 Literals

Literals are used for values such as strings, numbers, and dates.

A ***literal*** consists of two, three, or four components, as below:

1. A ***lexical form***, being an RDF string.
2. A ***datatype IRI***, being an IRI identifying a datatype that determines how the lexical form maps to a literal value.
3. If and only if the datatype IRI is `http://www.w3.org/1999/02/22-rdf-syntax-ns#langString` or `http://www.w3.org/1999/02/22-rdf-syntax-ns#dirLangString`, there is a non-empty ***language tag*** as defined by [*BCP47*]. The language tag *MUST* be well-formed according to section 2.2.9 of [*BCP47*], and *MUST* be treated accordingly, that is, in a case-insensitive manner. Two [*BCP47*]-complying strings that differ only by case represent the same language tag.
4. If and only if the datatype IRI is `http://www.w3.org/1999/02/22-rdf-syntax-ns#dirLangString`, there is a ***base direction*** that *MUST* be one of the following:
    - `ltr`, indicating that the initial text direction is set to left-to-right
    - `rtl`, indicating that the initial text direction is set to right-to-left

A literal is a ***language-tagged string*** if the language tag is present and the base direction is *not* present. A literal is a ***directional language-tagged string*** if both the language tag and the base direction are present.

Literal term equality: two literals are term-equal (the same RDF term) if and only if the following are all true:

- The two lexical forms compare equal, where this comparison is performed using case-sensitive matching (see description of string comparison in 2.2 Strings in RDF).
- The two datatype IRIs compare equal (per IRI equality).
- The two language tags are either both absent, or both present and compare equal, where this comparison is performed using ASCII case-insensitive matching (in contrast to the case-sensitive comparison of the lexical forms).
- The two base directions are either both absent, both `ltr`, or both `rtl`.

§ 3.4.1 Representation of Literals

Some concrete syntaxes support ***simple literals*** consisting of only a lexical form without any datatype IRI, language tag, or base direction. Simple literals are syntactic sugar for abstract syntax literals with the datatype IRI `http://www.w3.org/2001/XMLSchema#string` (which is commonly abbreviated as `xsd:string`).

Similarly, most concrete syntaxes represent language-tagged strings and directional language-tagged strings without the datatype IRI because it is always either `http://www.w3.org/1999/02/22-rdf-syntax-ns#langString` (`rdf:langString`) or `http://www.w3.org/1999/02/22-rdf-syntax-ns#dirLangString` (`rdf:dirLangString`), respectively.

Any string complying with [*BCP47*] *MAY* be used to represent a language tag in concrete syntaxes or implementations. Such strings *MAY* be case normalized (for example, by canonicalizing as defined by BCP 47 section 4.5). Alternatively, an implementation *MAY* preserve the case from the original representation, provided that it processes it in a case-insensitive manner.

> NOTE
> The treatment of language tags has changed between RDF 1.1 and RDF 1.2. In RDF 1.1, `"chat"@fr` and `"chat"@FR` represent two distinct terms, but implementations may replace either with the other via some form of normalization. In RDF 1.2, they represent the exact same literal, i.e., the case difference in the concrete syntax does not propagate into the abstract syntax. Since many RDF 1.1 implementations do normalize language tags internally, they will not be impacted by this change.

§ 3.4.2 Literal Value

The ***literal value*** associated with a literal is defined as follows.

- If the literal is a language-tagged string, then the literal value is a pair consisting of its lexical form and its language tag, in that order.
- If the literal is a directional language-tagged string, then the literal value is a tuple of its lexical form, its language tag, and its base direction, likewise in that order.
- If the literal's datatype is handled by an RDF implementation, then one of the following applies:
  - If the literal's lexical form is in the lexical space of the datatype, then the literal value is the result of applying the lexical-to-value mapping of the datatype to the lexical form.
  - Otherwise, the literal is ***ill-typed*** and no literal value can be associated with the literal. Such a case produces a semantic inconsistency, but it is not *syntactically* ill-formed. Implementations *SHOULD* accept ill-typed literals and produce RDF graphs from them. Implementations *MAY* produce warnings when encountering ill-typed literals.
- If the literal's datatype IRI is *not* handled by an RDF implementation, then the literal value is not defined by this specification. Implementations *SHOULD* accept literals with unknown datatype IRIs and produce RDF graphs from them.

It follows from the above that two literals can have the same value without being the same RDF term. For example:

```
"1"^^xsd:integer
"01"^^xsd:integer
```

denote the same value, but are not the same literal RDF term because their lexical forms differ.

§ 3.4.3 Initial Text Direction

This section is non-normative.

The base direction of a directional language-tagged string provides a means of establishing the initial direction of text, including text which is a mixture of right-to-left and left-to-right scripts. The Unicode Bidirectional Algorithm [*I18N-Glossary*] provides support for automatically rendering a sequence of characters in logical order, so that they are visually ordered as expected, but this is not sufficient to correctly render bidirectional text.

For example, some text with a language tag "`he`" might be displayed in a left-to-right context (such as an English web page) as פעילות הבינאום, W3C, which is incorrect. When provided to a user agent including base direction information (such as using HTML's `dir` attribute) it can then be correctly presented as:

W3C ,פעילות הבינאום

In the absence of an explicit initial text direction, the Unicode Bidirectional Algorithm detects the text direction from the content. This depends on the first strongly directional character in the text or on the context. To avoid spillover effects, users need to employ bidi isolation whenever text is inserted into a larger document. For example, "`<bdi lang="he">ספרים בינלאומיים!</bdi>`" displays the Hebrew characters in a right-to-left fashion — i.e., as "ספרים בינלאומיים!" — while they would be stored in memory as "מיימואלניב מירפס!"

§ 3.5 Blank Nodes

Blank nodes are disjoint from IRIs and literals. Otherwise, the set of possible blank nodes is arbitrary. RDF makes no reference to any internal structure of blank nodes.

Blank node equality: Two blank nodes are equal if and only if they are the same blank node.

NOTE

Blank node identifiers are local identifiers that are used in some concrete RDF syntaxes or RDF store implementations. They are always locally scoped to the file or RDF store, and are *not* persistent or portable identifiers for blank nodes. Blank node identifiers are *not* part of the RDF abstract data model, but are entirely dependent on the concrete syntax or implementation. The syntactic restrictions on blank node identifiers, if any, therefore also depend on the concrete RDF syntax or implementation. Implementations that handle blank node identifiers in concrete syntaxes need to be careful not to create the same blank node from multiple occurrences of the same blank node identifier except in situations where this is supported by the syntax.

The term "blank node label" is sometimes used informally as an alternative to the term blank node identifier. This alternative was also used in earlier versions of some RDF-related specifications such as [*SPARQL11-QUERY*]. In the interest of consistency, the use of this alternative term is discouraged now.

§ 3.6 Triple Terms

An RDF triple used as the object of another triple is called a ***triple term***. In a given RDF graph, a triple can appear as a triple term, an asserted triple, or both.

Triple term equality: Since triple terms are triples, equality of triple terms is the same as triple equality.

§ 3.7 Graph Comparison

This section introduces a notion of graph isomorphism for RDF graphs which is based on a mapping between RDF terms that maps blank nodes to blank nodes and is the identity function for IRIs and literals.

A function *M* from the set of all RDF terms into that same set is called an ***isomorphic RDF-term mapping*** if it is has all of the following properties:

- *M* is bijective.
- For every blank node *b*, *M*(*b*) is a blank node (but not necessarily the same as *b*).
- For every literal *lit*, *M*(*lit*) is *lit*.
- For every IRI *iri*, *M*(*iri*) is *iri*.
- For every triple term *tt* of the form (*s*, *p*, *o*), *M*(*tt*) is the triple term (*M*(*s*), *M*(*p*), *M*(*o*)).

Two RDF graphs *G* and *G*' are ***isomorphic*** (that is, they have the same form) if there exists an isomorphic RDF-term mapping *M* such that the triple (*s*, *p*, *o*) is in *G* if and only if the triple (*M*(*s*), *M*(*p*), *M*(*o*)) is in *G*'.

With this definition, *M* shows how each blank node in *G* can be replaced with a new blank node to give *G*'. Graph isomorphism is needed to support the RDF Test Cases [*RDF11-TESTCASES*] specification.

§ 4. RDF Datasets

An ***RDF dataset*** is a collection of RDF graphs, and comprises:

- Exactly one ***default graph***, being an RDF graph. The default graph does not have a name and *MAY* be empty.
- Zero or more ***named graphs***. Each named graph is a pair consisting of an IRI or a blank node (the ***graph name***), and an RDF graph. Graph names are unique within an RDF dataset.

Blank nodes can be shared between graphs in an RDF dataset.

NOTE

Despite the use of the word “name” in “named graph”, the graph name is not required to denote the graph. It is merely syntactically paired with the graph. RDF does not place any formal restrictions on what resource the graph name may denote, nor on the relationship between that resource and the graph. A discussion of different RDF dataset semantics can be found in [*RDF11-DATASETS*].

Some RDF dataset implementations do not track empty named graphs. Applications can avoid interoperability issues by not ascribing importance to the presence or absence of empty named graphs.

SPARQL version 1.2 [*SPARQL12-CONCEPTS*] uses the same concept of an RDF Dataset as RDF versions 1.1 and 1.2, in which the graph names of named graphs may be IRIs or blank nodes. In contrast, version 1.1 of the SPARQL Query Language [*SPARQL11-QUERY*] only allowed graph names to be IRIs.

§ 4.1 RDF Dataset Comparison

Two RDF datasets *D1* and *D2* (respectively, with default graphs *DG1* and *DG2* and sets *NG1* and *NG2* of named graphs) are ***dataset-isomorphic*** if and only if there exists an isomorphic RDF-term mapping *M* for which all of the following properties hold:

- The triple (s, p, o) is in *DG1* if and only if the triple $(M(s), M(p), M(o))$ is in *DG2*.
- The named graph (n, G) is in *NG1* if and only if there is a named graph (n', G') in *NG2* such that the following are true:
  - $M(n)$ is equal to n'.
  - For every triple $t=(s, p, o)$, it holds that t is in G if and only if the triple $(M(s), M(p), M(o))$ is in G'.

§ 4.2 Content Negotiation of RDF Datasets

This section is non-normative.

Web resources may have multiple representations that are made available via content negotiation [*WEBARCH*]. A representation may be returned in an RDF serialization format that supports the expression of both RDF datasets and RDF graphs. If an RDF dataset is returned and the consumer is expecting an RDF graph, the consumer is expected to use the RDF dataset's default graph.

§ 4.3 Dataset as a Set of Quads

This section is non-normative.

A ***quad*** is a triple associated with an optional graph name and is used when referring to triples within an RDF dataset.

A quad can be represented as a tuple composed of subject, predicate, object, and an optional graph name.

An RDF dataset can be considered to be a set of quads where quads with no graph name supply the triples of the default graph, and quads with the same graph name supply the triples of the named graph with that name.

> NOTE
>
> Although a quad without a graph name consists of the same three components as a triple, it is a distinct concept, as it specifically captures the notion of a triple within the default graph of an RDF dataset.

§ 5. Datatypes

Datatypes are used with RDF literals to represent values such as strings, numbers and dates. The datatype abstraction used in RDF is compatible with XML Schema [*XMLSCHEMA11-2*]. Any datatype definition that conforms to this abstraction *MAY* be used in RDF, even if not

defined in terms of XML Schema. RDF re-uses many of the XML Schema built-in datatypes, and defines three additional datatypes, `rdf:JSON`, `rdf:HTML`, and `rdf:XMLLiteral`.

A ***datatype*** consists of a lexical space, a value space and a lexical-to-value mapping, and is identified by one or more IRIs.

The ***lexical space*** of a datatype is a set of strings.

The ***lexical-to-value mapping*** of a datatype is a set of pairs whose first element belongs to the lexical space, and the second element belongs to the ***value space*** of the datatype. Each member of the lexical space is paired with exactly one value, and is a *lexical representation* of that value. The mapping can be seen as a function from the lexical space to the value space.

> NOTE
>
> Language-tagged strings have the datatype IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#langString` (commonly abbreviated as `rdf:langString`). No datatype is formally defined for this IRI because the definition of datatypes does not accommodate language tags in the lexical space. The value space associated with this datatype IRI is the set of all pairs that consist of a string and a language tag. Similarly, directional language-tagged strings `http://www.w3.org/1999/02/22-rdf-syntax-ns#dirLangString` (commonly abbreviated as `rdf:dirLangString`) also have a base direction in the value space. The value space associated with this datatype IRI is the set of all 3-tuples of a string, a language tag and a base direction.

For example, the XML Schema datatype `xsd:boolean`, where each member of the value space has two lexical representations, is defined as follows:

Lexical space:

{"true", "false", "1", "0"}

Value space:

{***true***, ***false***}

Lexical-to-value mapping

{ <"true", ***true***>, <"false", ***false***>, <"1", ***true***>, <"0", ***false***>, }

The literals that can be defined using this datatype are:

This table lists the literals of type xsd:boolean.

Literal	Value
<"true", xsd:boolean>	***true***
<"false", xsd:boolean>	***false***
<"1", xsd:boolean>	***true***
<"0", xsd:boolean>	***false***

§ 5.1 The XML Schema Built-in Datatypes

IRIs of the form `http://www.w3.org/2001/XMLSchema#xxx`, where *xxx* is the name of a datatype, denote the built-in datatypes defined in *W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes* [*XMLSCHEMA11-2*]. The XML Schema built-in types listed in the following table are the ***RDF-compatible XSD types***. Their use is *RECOMMENDED*.

Readers might note that the only safe datatypes for transferring binary information are `xsd:hexBinary` and `xsd:base64Binary`.

A list of the RDF-compatible XSD types, with short descriptions

	Datatype	Value space (informative)
Core types	xsd:string	Character strings
	xsd:boolean	true, false
	xsd:decimal	Arbitrary-precision decimal numbers
	xsd:integer	Arbitrary-size integer numbers
IEEE floating-point numbers	xsd:double	64-bit floating point numbers incl. ±Inf, ±0, NaN
	xsd:float	32-bit floating point numbers incl. ±Inf, ±0, NaN
Time and date	xsd:date	Dates (yyyy-mm-dd) with or without timezone
	xsd:time	Times (hh:mm:ss.sss…) with or without timezone

	xsd:dateTime	Date and time with or without timezone
	xsd:dateTimeStamp	Date and time with required timezone
Recurring and partial dates	xsd:gYear	Gregorian calendar year
	xsd:gMonth	Gregorian calendar month
	xsd:gDay	Gregorian calendar day of the month
	xsd:gYearMonth	Gregorian calendar year and month
	xsd:gMonthDay	Gregorian calendar month and day
Durations	xsd:duration	Duration of time
	xsd:yearMonthDuration	Duration of time (months and years only)
	xsd:dayTimeDuration	Duration of time (days, hours, minutes, seconds only)
Limited-range integer numbers	xsd:byte	-128…+127 (8 bit)
	xsd:short	-32768…+32767 (16 bit)
	xsd:int	-2147483648…+2147483647 (32 bit)
	xsd:long	-9223372036854775808… +9223372036854775807 (64 bit)
	xsd:unsignedByte	0…255 (8 bit)
	xsd:unsignedShort	0…65535 (16 bit)
	xsd:unsignedInt	0…4294967295 (32 bit)
	xsd:unsignedLong	0…18446744073709551615 (64 bit)
	xsd:positiveInteger	Integer numbers >0
	xsd:nonNegativeInteger	Integer numbers ≥0
	xsd:negativeInteger	Integer numbers <0
	xsd:nonPositiveInteger	Integer numbers ≤0
Encoded	xsd:hexBinary	Hex-encoded binary data

binary data	`xsd:base64Binary`	Base64-encoded binary data
Miscellaneous XSD types	`xsd:anyURI`	Resolved or relative URI and IRI references
	`xsd:language`	Language tags per [*BCP47*]
	`xsd:normalizedString`	Whitespace-normalized strings
	`xsd:token`	Tokenized strings
	`xsd:NMTOKEN`	XML NMTOKENs
	`xsd:Name`	XML Names
	`xsd:NCName`	XML NCNames

The lexical-to-value mapping for `xsd:float` and `xsd:double` *MUST* use a method consistent with doubleLexicalMap, which *MUST* strictly conform to the rounding method described in floatPtRound [*XMLSCHEMA11-2*].

The other built-in XML Schema datatypes are unsuitable for various reasons and *SHOULD NOT* be used:

- `xsd:QName` and `xsd:ENTITY` require an enclosing XML document context.
- `xsd:ID` and `xsd:IDREF` are for cross references within an XML document.
- `xsd:NOTATION` is not intended for direct use.
- `xsd:IDREFS`, `xsd:ENTITIES` and `xsd:NMTOKENS` are sequence-valued datatypes which do not fit the RDF datatype model.

NOTE

The value spaces of `xsd:double` and `xsd:float` do not include all decimal numbers. For every literal of either of these two datatypes, the value of the literal is a value that can be represented as an IEEE 754-2008 binary floating point representation of the corresponding precision. For instance, the literal with lexical form `"0.1"` and datatype `xsd:float` denotes the number `0.100000001490116119384765625`. Rather than `xsd:double` or `xsd:float`, the datatype `xsd:decimal` can be used to accurately capture arbitrary decimal numbers.

§ 5.2 Datatype IRIs

Datatypes are identified by IRIs.

If any IRI of the form `http://www.w3.org/2001/XMLSchema#xxx` is handled by an RDF implementation, it *MUST* refer to the RDF-compatible XSD type named `xsd:xxx` for every XSD type listed in section 5.1.

The datatypes identified by the three IRIs below are defined in Appendix A. Additional Datatypes:

- The IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#XMLLiteral` refers to the datatype `rdf:XMLLiteral`.
- The IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#HTML` refers to the datatype `rdf:HTML`.
- The IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#JSON` refers to the datatype `rdf:JSON`.

RDF implementations are not required to handle all datatypes. Any literal typed with a datatype not handled by an RDF implementation is treated just like an unknown IRI, i.e., as referring to an unknown thing. Applications *MAY* give a warning message if they are unable to determine the referent of an IRI used in a typed literal. RDF implementations *SHOULD NOT* reject a literal with an unknown datatype as either a syntactic or semantic error.

Other specifications *MAY* impose additional constraints on datatype IRIs, for example, require support for certain datatypes.

> NOTE
>
> Semantic extensions of RDF might choose to recognize other datatype IRIs and require each of them to refer to a fixed datatype. See *RDF 1.2 Semantics* [*RDF12-SEMANTICS*] for more information on semantic extensions.

NOTE

The Web Ontology Language [*OWL2-OVERVIEW*] offers facilities for formally defining custom datatypes that can be used with RDF. Furthermore, a practice for identifying user-defined simple XML Schema datatypes is suggested in [*SWBP-XSCH-DATATYPES*]. RDF implementations are not required to support either of these facilities.

NOTE

In RDF 1.1, *Recognized datatype IRIs* were defined in RDF Concepts, overlapping with RDF Semantics, "recognizing" datatype IRIs for semantic extensions.

§ 6. Fragment Identifiers

This section is non-normative.

RDF uses IRIs, which may include ***fragment identifiers***, as resource identifiers. The semantics of fragment identifiers is defined in RFC 3986 [*RFC3986*]: They identify a secondary resource that is usually a part of, a view of, defined in, or described in the primary resource, and the precise semantics depend on the set of representations that might result from a retrieval action on the primary resource.

This section discusses the handling of fragment identifiers in representations that encode RDF graphs.

In RDF-bearing representations of a primary resource, e.g., `<https://example.com/foo>`, the secondary resource identified by a fragment identifier, e.g., `bar`, is the resource denoted by the full IRI in the RDF graph, which would be `<https://example.com/foo#bar>` in this case. Since IRIs in RDF graphs can denote anything, this can be something external to the representation, or even external to the web.

In this way, the RDF-bearing representation acts as an intermediary between the web-accessible primary resource and some set of possibly non-web or abstract entities that the RDF graph may describe.

In cases where other specifications constrain the semantics of fragment identifiers in RDF-bearing representations, the encoded RDF graph should use fragment identifiers in a way that is consistent with these constraints. For example, in an HTML+RDFa document [*HTML-RDFA*], a fragment identifier such as `chapter1` may identify a document section via the semantics of HTML's `@name` or `@id` attributes. Such an IRI, e.g., `<#chapter1>`, should then be taken to denote that same section in any RDFa-encoded triples within the same document. Similarly, fragment identifiers should be used consistently in resources with multiple representations that are made available via content negotiation [*WEBARCH*]. For example, if the fragment identifier `chapter1` identifies a document section in an HTML representation of the primary resource, then the IRI `<#chapter1>` should be taken to denote that same section in all RDF-bearing representations of the same primary resource.

§ 7. Generalizations of RDF Triples, Graphs, and Datasets

This section is non-normative.

It is sometimes convenient to loosen the requirements on RDF triples. For example, the completeness of the RDFS entailment rules is easier to show with a notion of symmetric RDF triples.

§ 7.1 Symmetric RDF

A ***symmetric RDF triple*** allows the subject to be any RDF term that is allowed in the object position, one of an IRI, a blank node, a literal, or a triple term (which may itself be a symmetric RDF triple). A ***symmetric RDF graph*** is a set of symmetric RDF triples. A ***symmetric RDF dataset*** comprises a distinguished symmetric RDF graph, and zero or more pairs that each associate an IRI or a blank node with a symmetric RDF graph.

Symmetric RDF triples, graphs, and datasets differ from standard, normative RDF triples, graphs, and datasets only by allowing IRIs, blank nodes, literals, and triple terms in the subject and object positions.

7.2 Generalized RDF

A ***generalized RDF triple*** is a triple having a subject, a predicate, and an object, where each can be an IRI, a blank node, a literal, or a triple term (which may itself be a generalized RDF triple). A ***generalized RDF graph*** is a set of generalized RDF triples. A ***generalized RDF dataset*** comprises a distinguished generalized RDF graph, and zero or more pairs each associating an IRI, a blank node, a literal, or a triple term (which may itself be a generalized RDF triple), to a generalized RDF graph.

Generalized RDF triples, graphs, and datasets differ from standard, normative RDF triples, graphs, and datasets only by allowing IRIs, blank nodes, literals, and triple terms to appear in any position, i.e., as subject, predicate, object, or graph name.

> NOTE
>
> Any user of symmetric or generalized RDF triples, graphs, or datasets needs to be aware that these notions are non-standard extensions of RDF, and their use may cause interoperability problems. There is no requirement for any RDF tool to accept, process, or produce anything beyond standard normative RDF triples, graphs, and datasets.

A. Additional Datatypes

This section defines additional datatypes that RDF implementations *MAY* support.

A.1 The `rdf:HTML` Datatype

RDF provides for HTML content as a possible literal value. This allows markup in literal values. Such content is indicated in an RDF graph using a literal whose datatype is set to ***`rdf:HTML`***.

The `rdf:HTML` datatype is defined as follows:

The IRI denoting this datatype
is `http://www.w3.org/1999/02/22-rdf-syntax-ns#HTML`.

The value space
is the set of DOM `DocumentFragment` nodes [*DOM*]. Two `DocumentFragment` nodes *node* and *otherNode* are considered equal if and only if the DOM method `node.isEqualNode(otherNode)` [*DOM*] returns `true`.

The lexical-to-value mapping
Each member of the lexical space is associated with the result of applying the following algorithm:

- Let `domnodes` be the list of DOM nodes [*DOM*] that result from applying the HTML fragment parsing algorithm [*HTML5*] to the input string, without a context element.
- Let `domfrag` be a DOM `DocumentFragment` [*DOM*] whose `childNodes` attribute is equal to `domnodes`
- Return `domfrag.normalize()`.

> NOTE
>
> Any language annotation (`lang="…"`), text directionality annotation (`dir="…"`), or XML namespaces (`xmlns`) desired in the HTML content must be included explicitly in the HTML literal. Relative URLs in attributes such as `href` do not have a well-defined base URL and are best avoided. RDF applications may use additional equivalence relations, such as that which relates an `xsd:string` with an `rdf:HTML` literal corresponding to a single text node of the same string.

§ A.2 The `rdf:XMLLiteral` Datatype

RDF provides for XML content as a possible literal value. Such content is indicated in an RDF graph using a literal whose datatype is set to ***`rdf:XMLLiteral`***.

The `rdf:XMLLiteral` datatype is defined as follows:

The IRI denoting this datatype

is `http://www.w3.org/1999/02/22-rdf-syntax-ns#XMLLiteral`.

The lexical space
is the set of all strings which are well-balanced, self-contained XML content [*XML11*]; and for which embedding between an arbitrary XML start tag and an end tag yields a document conforming to *Namespaces in XML 1.0 (Third Edition)* [*XML-NAMES*].

The value space
is the set of DOM `DocumentFragment` nodes [*DOM*]. Two `DocumentFragment` nodes *node* and *otherNode* are considered equal if and only if the DOM method `node.isEqualNode(otherNode)` returns `true`.

The lexical-to-value mapping
Each member of the lexical space is associated with the result of applying the following algorithm:

- Let `domfrag` be a DOM `DocumentFragment` node [*DOM*] corresponding to the input string.
- Return `domfrag.normalize()`.

> NOTE
>
> Any XML namespace declarations (`xmlns`), language annotation (`xml:lang`) or base URI declarations (`xml:base`) desired in the XML content must be included explicitly in the XML literal. Note that some concrete RDF syntaxes may define mechanisms for inheriting them from the context (e.g., `@parseType="literal"` in RDF/XML [*RDF12-XML*].

§ A.3 The `rdf:JSON` Datatype

RDF provides for JSON content as a possible literal value. This includes allowing markup in literal values. Such content is indicated in an RDF graph as a literal whose datatype is set to ***`rdf:JSON`***.

The `rdf:JSON` datatype is defined as follows:

The IRI denoting this datatype
is `http://www.w3.org/1999/02/22-rdf-syntax-ns#JSON`.

The lexical space

is the set of all RDF strings that conform to the JSON Grammar as described in Section 2 JSON Grammar of [*RFC8259*], which also conform to the requirements of *The I-JSON Message Format* [*RFC7493*].

> NOTE
> *The JavaScript Object Notation (JSON) Data Interchange Format* [*RFC8259*] allows strings to include surrogate code points not allowed in RDF strings, which are also excluded in [*RFC7493*], thus the lexical representation of JSON literals excludes those including surrogate code points.

The value space

is the smallest set containing strings, numbers (**xsd:double**), finite unordered maps mapping strings to values in the value space, lists of values in the value space, and literal values (`true`, `false`, and `null`) from *Infra Standard* [*INFRA*] and *W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes* [*XMLSCHEMA11-2*].

> NOTE
>
> The value space of finite unordered maps and lists does not include values having themselves as members, which cannot be represented in JSON.

Two values are considered equal if and only if they are the same element of the value space.

The lexical-to-value mapping

maps every element of the lexical space to the result of parsing it into a string, number (**xsd:double**), finite unordered map, list, or literal value (`true`, `false`, and `null`).

- A JSON Object is mapped to a finite unordered map by transforming each object member into a map entry with the key taken from the member name and value taken by performing this mapping to the member value.
- A JSON Array is mapped to a list such that this list contains as many elements as the JSON Array and, for every position *i* in the array, the element at the *i*-th position in the list is the value that results from applying this mapping to the *i*-th element of the array.
- A JSON Number is mapped to an **xsd:double** using a method consistent with doubleLexicalMap, which *MUST* strictly conform to the rounding method described

in floatPtRound [*XMLSCHEMA11-2*].

> NOTE
> Some numbers cannot be represented as finite **xsd:double** values and may map to `+INF` or `-INF`. Such values cannot be represented as JSON Numbers, limiting the ability to serialize such values back to JSON.

- A JSON String is mapped to a string after converting any escape sequences to the associated Unicode code point.
- A JSON literal name maps JSON `true`, `false`, and `null` values to [*INFRA*] `true`, `false`, and `null` values, respectively.

> NOTE
>
> The finite unordered maps can be implemented with ordered maps [*INFRA*] by systematically sorting key-value pairs by key (using Unicode code point order). This ensures that lexical forms that differ only in the order of object members (e.g., `{"a": "b", "c": "d"}` and `{"c": "d", "a": "b"}`) are mapped to the same value.

§ B. Replacing Blank Nodes with IRIs

Blank nodes do not have identifiers in the RDF abstract data model. The blank node identifiers introduced by some concrete syntaxes have only local scope and are purely an artifact of the serialization.

In situations where stronger identification is needed, systems *MAY* systematically replace some or all of the blank nodes in an RDF graph with IRIs. Systems wishing to do this *SHOULD* mint a new, globally unique IRI (a ***Skolem IRI***) for each blank node so replaced.

This transformation produces a new graph which entails the original one (see also Skolemization). In this context it is important to note that Skolem IRIs and blank nodes are different in nature: while IRIs directly denote resources, blank nodes only indicate the existence of a resource. By producing Skolem IRIs, we create concrete witnesses of this

existence. As Skolem IRIs are not local, this permits other graphs to subsequently use them, which is not possible with blank nodes.

Systems may wish to mint Skolem IRIs in such a way that they can recognize the IRIs as having been introduced solely to replace blank nodes. In some contexts, this allows a system to map Skolem IRIs back to blank nodes if needed.

Systems that want Skolem IRIs to be recognizable outside of the system boundaries *SHOULD* use a well-known IRI [*RFC8615*] with the registered name `genid`. This is an IRI that uses the HTTP or HTTPS scheme, or another scheme that has been specified to use well-known IRIs, and whose path component starts with `/.well-known/genid/`.

For example, the authority responsible for the domain `example.com` could mint the following recognizable Skolem IRI:

```
http://example.com/.well-known/genid/d26a2d0e98334696f4ad70a677abc1f6
```

> NOTE
>
> RFC 8615 [*RFC8615*] only specifies well-known URIs, not IRIs. For the purpose of this document, a well-known IRI is any IRI that results in a well-known URI after IRI-to-URI mapping [*RFC3987*].

§ C. Privacy Considerations

This section is non-normative.

RDF is used to express arbitrary application data, which may include the expression of personally identifiable information (PII) or other information which could be considered sensitive. Authors publishing such information are advised to carefully consider the needs and use of publishing such information, as well as the applicable regulations for the regions where the data is expected to be consumed and potentially revealed (e.g., GDPR, CCPA, others), particularly whether authorization measures are needed for access to the data.

§ D. Security Considerations

This section is non-normative.

The RDF Abstract Data Model is not used directly for conveying information. Concrete serialization forms are specifically intended to do so.

Applications can evaluate given data to infer more assertions or to dereference IRIs, invoking the security considerations of the scheme for that IRI. Note in particular, the privacy issues in [*RFC3023*] section 10 for HTTP IRIs. Data obtained from an inaccurate or malicious data source may lead to inaccurate or misleading conclusions, as well as the dereferencing of unintended IRIs. Care must be taken to align the trust in consulted resources with the sensitivity of the intended use of the data; inferences of potential medical treatments would likely require different trust than inferences for trip planning.

RDF is used to express arbitrary application data; security considerations will vary by domain of use. Security tools and protocols applicable to text (for example, PGP encryption, checksum validation, password-protected compression) can also be used on RDF documents. Security/privacy protocols ought to be imposed which reflect the sensitivity of the embedded information.

RDF can express data which is presented to the user, such as RDF Schema labels. Applications rendering strings retrieved from untrusted RDF documents, or using unescaped characters, ought to use warnings and other appropriate means to limit the possibility that malignant strings might be used to mislead the reader. The security considerations in the media type registration for XML ([*RFC3023*] section 10) provide additional guidance around the expression of arbitrary data and markup.

RDF uses IRIs as term identifiers. Applications interpreting data expressed in RDF ought to address the security issues of *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8, as well as *Uniform Resource Identifier (URI): Generic Syntax* [*RFC3986*] Section 7.

Multiple IRIs can have the same appearance. Characters in different scripts can look similar (for instance, a Cyrillic "o" can appear similar to a Latin "o"). A character followed by combining characters can have the same visual representation as another character (for example, LATIN SMALL LETTER "E" followed by COMBINING ACUTE ACCENT has the

same visual representation as LATIN SMALL LETTER "E" WITH ACUTE). Any person or application that is writing or interpreting data in RDF must take care to use the IRI that matches the intended semantics, and avoid IRIs that may look similar. Further information about matching visually similar characters can be found in *Unicode Security Considerations* [*UNICODE-SECURITY*] and *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8.

Comparing graphs, or reasoning with them, often relies on computing *(sub)graph isomorphism*, which is known to be computationally complex in the worst case. Querying graphs can also involve computationally complex operations. This means that malicious graphs can be constructed to cause RDF implementations to stall or run out of memory. Implementations processing graphs from untrusted sources are expected to provide mitigations; examples are given in the section on Dataset Poisoning in [*RDF-CANON*].

> NOTE
>
> These considerations are a more generic form of Security Considerations for [*RDF12-TURTLE*], [*RDF12-TRIG*], [*RDF12-N-TRIPLES*], and [*RDF12-N-QUADS*].

§ E. Internationalization Considerations

This section is non-normative.

Unicode [*UNICODE*] provides a mechanism for signaling direction within a string (see Unicode Bidirectional Algorithm [*I18N-Glossary*]). RDF provides a mechanism for specifying the base direction of a directional language-tagged string to signal the initial text direction of a string. For most human language strings, but particularly for those whose base direction cannot be accurately determined from the string content, is it valuable to have an external indicator in order to get the proper display and isolation of the value. One example of such an indicator is the [*HTML*] dir attribute; see [*STRING-META*].

JSON-LD 1.1 [*JSON-LD11*] introduced the i18n namespace to use a datatype to specify both the base direction an language tag of an RDF literal.

§ F. IRI Grammar

This section is non-normative.

The following [*ABNF*] grammar applies the changes from [*RFC3987*] and [*RFC6874*] to the section Collected ABNF for URI of [*RFC3986*] to give a consolidated grammar for IRIs.

This is provided for convenience only. If it differs from definitions in [*RFC3986*], [*RFC3987*], or any subsequent updates, then those definitions should be used.

```
IRI            = scheme ":" ihier-part [ "?" iquery ] [ "#" ifragmen

ihier-part     = "//" iauthority ipath-abempty
               / ipath-absolute
               / ipath-rootless
               / ipath-empty

IRI-reference  = IRI / irelative-ref

absolute-IRI   = scheme ":" ihier-part [ "?" iquery ]

irelative-ref  = irelative-part [ "?" iquery ] [ "#" ifragment ]

irelative-part = "//" iauthority ipath-abempty
               / ipath-absolute
               / ipath-noscheme
               / ipath-empty

scheme         = ALPHA *( ALPHA / DIGIT / "+" / "-" / "." )

iauthority     = [ iuserinfo "@" ] ihost [ ":" port ]
iuserinfo      = *( iunreserved / pct-encoded / sub-delims / ":" )
ihost          = IP-literal / IPv4address / ireg-name
port           = *DIGIT

IP-literal     = "[" ( IPv6address / IPv6addrz / IPvFuture  ) "]"

ZoneID         = 1*( unreserved / pct-encoded )
```

```
IPv6addrz      = IPv6address "%25" ZoneID

IPvFuture      = "v" 1*HEXDIG "." 1*( unreserved / sub-delims / ":"

IPv6address    =                            6( h16 ":" ) ls32
               /                       "::" 5( h16 ":" ) ls32
               / [               h16 ] "::" 4( h16 ":" ) ls32
               / [ *1( h16 ":" ) h16 ] "::" 3( h16 ":" ) ls32
               / [ *2( h16 ":" ) h16 ] "::" 2( h16 ":" ) ls32
               / [ *3( h16 ":" ) h16 ] "::"    h16 ":"   ls32
               / [ *4( h16 ":" ) h16 ] "::"              ls32
               / [ *5( h16 ":" ) h16 ] "::"              h16
               / [ *6( h16 ":" ) h16 ] "::"

h16            = 1*4HEXDIG
ls32           = ( h16 ":" h16 ) / IPv4address
IPv4address    = dec-octet "." dec-octet "." dec-octet "." dec-octet

dec-octet      = DIGIT                 ; 0-9
               / %x31-39 DIGIT         ; 10-99
               / "1" 2DIGIT            ; 100-199
               / "2" %x30-34 DIGIT     ; 200-249
               / "25" %x30-35          ; 250-255

ireg-name      = *( iunreserved / pct-encoded / sub-delims )

ipath          = ipath-abempty   ; begins with "/" or is empty
               / ipath-absolute  ; begins with "/" but not "//"
               / ipath-noscheme  ; begins with a non-colon segment
               / ipath-rootless  ; begins with a segment
               / ipath-empty     ; zero characters

ipath-abempty  = *( "/" isegment )
ipath-absolute = "/" [ isegment-nz *( "/" isegment ) ]
ipath-noscheme = isegment-nz-nc *( "/" isegment )
ipath-rootless = isegment-nz *( "/" isegment )
ipath-empty    = 0

isegment       = *ipchar
isegment-nz    = 1*ipchar
isegment-nz-nc = 1*( iunreserved / pct-encoded / sub-delims / "@" )
```

```
                ; non-zero-length segment without any colon ":"

ipchar          = iunreserved / pct-encoded / sub-delims / ":" / "@"

iquery          = *( ipchar / iprivate / "/" / "?" )

ifragment       = *( ipchar / "/" / "?" )

iunreserved     = ALPHA / DIGIT / "-" / "." / "_" / "~" / ucschar

ucschar         = %xA0-D7FF / %xF900-FDCF / %xFDF0-FFEF
                / %x10000-1FFFD / %x20000-2FFFD / %x30000-3FFFD
                / %x40000-4FFFD / %x50000-5FFFD / %x60000-6FFFD
                / %x70000-7FFFD / %x80000-8FFFD / %x90000-9FFFD
                / %xA0000-AFFFD / %xB0000-BFFFD / %xC0000-CFFFD
                / %xD0000-DFFFD / %xE1000-EFFFD

iprivate        = %xE000-F8FF / %xF0000-FFFFD / %x100000-10FFFD

pct-encoded     = "%" HEXDIG HEXDIG

unreserved      = ALPHA / DIGIT / "-" / "." / "_" / "~"
reserved        = gen-delims / sub-delims
gen-delims      = ":" / "/" / "?" / "#" / "[" / "]" / "@"
sub-delims      = "!" / "$" / "&" / "'" / "(" / ")"
                / "*" / "+" / "," / ";" / "="
```

The ABNF can also be accessed directly from iri-grammar.abnf.

§ G. Acknowledgments

This section is non-normative.

§ G.1 Acknowledgments for RDF 1.0

This section is non-normative.

The editors of the original version of the spec were Graham Klyne (Nine by Nine) and Jeremy J. Carroll (Hewlett Packard Labs).

This document contains a significant contribution from Pat Hayes, Sergey Melnik and Patrick Stickler, under whose leadership was developed the framework described in the RDF family of specifications for representing datatyped values, such as integers and dates.

The editors acknowledge valuable contributions from the following: Frank Manola, Pat Hayes, Dan Brickley, Jos de Roo, Dave Beckett, Patrick Stickler, Peter F. Patel-Schneider, Jerome Euzenat, Massimo Marchiori, Tim Berners-Lee, Dave Reynolds and Dan Connolly.

Jeremy Carroll thanks Oreste Signore, his host at the W3C Office in Italy and Istituto di Scienza e Tecnologie dell'Informazione "Alessandro Faedo", part of the Consiglio Nazionale delle Ricerche, where Jeremy is a visiting researcher.

This document is a product of extended deliberations by the RDFcore Working Group, whose members have included: Art Barstow (W3C), Dave Beckett (ILRT), Dan Brickley (ILRT), Dan Connolly (W3C), Jeremy Carroll (Hewlett Packard), Ron Daniel (Interwoven Inc), Bill dehOra (InterX), Jos De Roo (AGFA), Jan Grant (ILRT), Graham Klyne (Nine by Nine), Frank Manola (MITRE Corporation), Brian McBride (Hewlett Packard), Eric Miller (W3C), Stephen Petschulat (IBM), Patrick Stickler (Nokia), Aaron Swartz (HWG), Mike Dean (BBN Technologies / Verizon), R. V. Guha (Alpiri Inc), Pat Hayes (IHMC), Sergey Melnik (Stanford University) and Martyn Horner (Profium Ltd).

This specification also draws upon an earlier RDF Model and Syntax document edited by Ora Lassilla and Ralph Swick, and RDF Schema edited by Dan Brickley and R. V. Guha. RDF and RDF Schema Working Group members who contributed to this earlier work are: Nick Arnett (Verity), Tim Berners-Lee (W3C), Tim Bray (Textuality), Dan Brickley (ILRT / University of Bristol), Walter Chang (Adobe), Sailesh Chutani (Oracle), Dan Connolly (W3C), Ron Daniel (DATAFUSION), Charles Frankston (Microsoft), Patrick Gannon (CommerceNet), R. V. Guha (Epinions, previously of Netscape Communications), Tom Hill (Apple Computer), Arthur van Hoff (Marimba), Renato Iannella (DSTC), Sandeep Jain (Oracle), Kevin Jones, (InterMind), Emiko Kezuka (Digital Vision Laboratories), Joe Lapp (webMethods Inc.), Ora Lassila (Nokia Research Center), Andrew Layman (Microsoft), Ralph LeVan (OCLC), John McCarthy (Lawrence Berkeley National Laboratory), Chris McConnell (Microsoft), Murray Maloney (Grif), Michael Mealling (Network Solutions), Norbert Mikula (DataChannel), Eric Miller

(OCLC), Jim Miller (W3C, emeritus), Frank Olken (Lawrence Berkeley National Laboratory), Jean Paoli (Microsoft), Sri Raghavan (Digital/Compaq), Lisa Rein (webMethods Inc.), Paul Resnick (University of Michigan), Bill Roberts (KnowledgeCite), i Tsuyoshi Sakata (Digital Vision Laboratories), Bob Schloss (IBM), Leon Shklar (Pencom Web Works), David Singer (IBM), Wei (William) Song (SISU), Neel Sundaresan (IBM), Ralph Swick (W3C), Naohiko Uramoto (IBM), Charles Wicksteed (Reuters Ltd.), Misha Wolf (Reuters Ltd.) and Lauren Wood (SoftQuad).

§ G.2 Acknowledgments for RDF 1.1

This section is non-normative.

The editors of the RDF 1.1 version of the spec were Richard Cyganiak (DERI), David Wood (3 Round Stones), and Markus Lanthaler (Graz University of Technology).

The editors acknowledge valuable contributions from Thomas Baker, Tim Berners-Lee, David Booth, Dan Brickley, Gavin Carothers, Jeremy Carroll, Pierre-Antoine Champin, Dan Connolly, John Cowan, Martin J. Dürst, Alex Hall, Steve Harris, Sandro Hawke, Pat Hayes, Ivan Herman, Peter F. Patel-Schneider, Addison Phillips, Eric Prud'hommeaux, Nathan Rixham, Andy Seaborne, Leif Halvard Silli, Guus Schreiber, Dominik Tomaszuk, and Antoine Zimmermann.

The membership of the RDF Working Group included Thomas Baker, Scott Bauer, Dan Brickley, Gavin Carothers, Pierre-Antoine Champin, Olivier Corby, Richard Cyganiak, Souripriya Das, Ian Davis, Lee Feigenbaum, Fabien Gandon, Charles Greer, Alex Hall, Steve Harris, Sandro Hawke, Pat Hayes, Ivan Herman, Nicholas Humfrey, Kingsley Idehen, Gregg Kellogg, Markus Lanthaler, Arnaud Le Hors, Peter F. Patel-Schneider, Eric Prud'hommeaux, Yves Raimond, Nathan Rixham, Guus Schreiber, Andy Seaborne, Manu Sporny, Thomas Steiner, Ted Thibodeau, Mischa Tuffield, William Waites, Jan Wielemaker, David Wood, Zhe Wu, and Antoine Zimmermann.

§ G.3 Acknowledgments for RDF 1.2

This section is non-normative.

In addition to the editors, the following people have contributed to this specification: Denis Ah-Kang, doerthe, Dominik Tomaszuk, Enrico Franconi, james anderson, Niklas Lindström, Peter F. Patel-Schneider, Ruben Taelman, Sarven Capadisli, Ted Thibodeau Jr, Thomas Tanon, and William Van Woensel

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

> EDITOR'S NOTE
>
> Recognize members of the Task Force? Not an easy to find list of contributors.

§ H. Changes between RDF 1.1 and RDF 1.2

This section is non-normative.

- Added 1.10 RDF Version Announcement for announcing RDF versions in RDF documents via the HTTP `Content-Type` header and/or syntax.
- Added 4.3 Dataset as a Set of Quads for informative definition of a quad.
- Added 1.5 Triple Terms and Reification and definitions for triple term and asserted triple and extended the definition of RDF triple to permit triple terms as objects. Also defines reifier and reifying triple.

- Added the base direction component as part of a literal, and a description of its use in 3.4.3 Initial Text Direction.
- Improved the use of IRI terminology, and added F. IRI Grammar. This improves the language using relative IRI references and clarifies that, in the abstract syntax, IRIs are resolved, avoiding the incorrect use of "absolute IRI".
- Changed reference from DOM4, which was not a recommendation at the time, to [*DOM*], making the definitions of `rdf:HTML` and `rdf:XMLLiteral` datatypes normative.
- Added A. Additional Datatypes and moved the sections about the `rdf:HTML` and `rdf:XMLLiteral` datatypes to this appendix.
- Added the `rdf:JSON` datatype, the definition of which is adopted from Section 10.2 The `rdf:JSON` Datatype in [*JSON-LD11*]. Note that the value space defined here updates the value space of the `rdf:JSON` datatype defined in *JSON-LD 1.1* [*JSON-LD11*]
- Clarify Unicode terminology, using Unicode code points, and restriction to the XML Char production. Also removes obsolete recommendations for the use of Normalization Form C in literals. Adds a definition of string that can be used in other RDF documents.
- Minor edit to improve the example about distinguishing literals, IRIs, and blank nodes in 3.1 Triples.
- Implementations were previously allowed to normalize language tags to lower case, which made it ambiguous whether two literals with language tags that differed only by case represented the same literal, or distinct literals. RDF 1.2 requires that language tags be case-insensitively unique but does not specify the common formatting to be used. Two literals with the same lexical form and language tags that differ only by case are the same literal. Implementations can either follow the advice to normalize to lower case, use the recommended BCP47 format, or do something else, as long it is performed consistently.
- Added explicit definitions of blank node equality, RDF term equality, and triple equality.
- Removed the section on the canonical mapping for the `rdf:XMLLiteral` datatype.
- Refer to the definition and discussion of RDF Semantics, "recognizing" datatype IRIs, instead of *Recognized datatype IRIs*.
- The informal terminology "RDF processor" has been removed.

NOTE

A detailed overview of the differences between RDF versions 1.1 and 1.2 can be found in *What's New in RDF 1.2* [*RDF12-NEW*].

I. Index

I.1 Terms defined by this specification

abstract syntax §1.9

appearing §3.2

asserted §3.

base direction §3.4

base IRI §3.3.1

Basic conformance §2.

basic RDF terms §3.2

Blank node equality §3.5

Blank node identifiers §3.5

Blank nodes §3.5

concrete RDF syntax §1.9

dataset-isomorphic §4.1

datatype §5.

datatype IRI §3.4

default graph §4.

denotes §1.2

directional language-tagged string §3.4

Entailment §1.8

Equivalence §1.8

fragment identifiers §6.

Full conformance §2.

generalized RDF dataset §7.2

generalized RDF graph §7.2

generalized RDF triple §7.2

graph name §4.

ground §3.2

ill-typed §3.4.2

Inconsistency §1.8

IRI §3.3

IRI equality §3.3

IRI references §3.3

isomorphic §3.7

isomorphic RDF-term mapping §3.7

language tag §3.4

language-tagged string §3.4

lexical form §3.4

lexical space §5.

lexical-to-value mapping §5.

literal §3.4

Literal term equality §3.4

literal value §3.4.2

named graphs §4.

namespace §1.4

namespace IRI §1.4

namespace prefix §1.4

nodes §3.1

object §3.1

predicate §3.1

property §1.2

proposition §1.8

quad §4.3

RDF dataset §4.

RDF document §1.9

RDF graph §3.

RDF Reference IRI §3.3.1

RDF source §1.6

RDF statement §1.2

RDF string §2.2

RDF term equality §3.2

RDF terms §3.2

RDF triple §3.1

RDF vocabulary §1.4

RDF-compatible XSD types §5.1

`rdf:HTML` §A.1

`rdf:JSON` §A.3

`rdf:XMLLiteral` §A.2

referent §1.3

reifier §1.5

reifying triple §1.5

relative IRI references §3.3.1

resources §1.2

simple literals §3.4.1

Skolem IRI §B.

subject §3.1

symmetric RDF dataset §7.1

symmetric RDF graph §7.1

symmetric RDF triple §7.1

transparent §1.5

triple annotation §1.5

Triple equality §3.1

triple term §3.6

URIs §3.3

value space §5

value space §3.

version label §2.1

§ I.2 Terms defined by reference

[*BCP47*] defines the following:

BCP 47 section 4.5

section 2.2.9

[*DID-CORE*] defines the following:

DID syntax

[*DOM*] defines the following:

DocumentFragment

DOM nodes

`isEqualNode(otherNode)` (for `Node`)

`normalize()` (for `Node`)

[*HTML*] defines the following:

dir attribute

[*HTML5*] defines the following:

HTML fragment parsing algorithm

[*I18N-GLOSSARY*] defines the following:

ASCII case-insensitive matching

bidi isolation

case-sensitive matching

code units

Normalization Form C

spillover effects

surrogate code points

Unicode Bidirectional Algorithm

Unicode character encoding

Unicode code points

Unicode scalar values

[*INFRA*] defines the following:

key

lists

map entry

null

ordered maps

true, false

value

[*JSON-LD11*] defines the following:

i18n namespace

Section 10.2 The rdf:JSON Datatype

[*OWL2-OVERVIEW*] defines the following:

OWL 2

[*OWL2-SYNTAX*] defines the following:

custom datatypes

[*RDF-CANON*] defines the following:

Dataset Poisoning

[*RDF12-SEMANTICS*] defines the following:

entailment regime

interpretation

RDF Semantics, "recognizing"

semantic extensions

Skolemization

[*RDF12-XML*] defines the following:

@parseType="literal"

[*RFC3986*] defines the following:

base IRI can be established

Collected ABNF for URI

fragment identifier

IRI scheme

path component

relative reference

resolved

resolved against

[*RFC3987*] defines the following:

[*RFC3987*] defines the following:

- section 1.3
- section 3.1
- Section 5
- section 5.3.1

[*RFC7230*] defines the following:

- HTTP URI scheme
- http-URI
- scheme rules for HTTP/HTTPS

[*RFC8259*] defines the following:

- JSON Array
- JSON Grammar
- JSON literal name
- JSON Number
- JSON Object
- JSON String

[*SPARQL12-QUERY*] defines the following:

- the same concept of an RDF Dataset

[*SWBP-N-ARYRELATIONS*] defines the following:

- indirectly expressed in RDF

[*SWBP-XSCH-DATATYPES*] defines the following:

- user-defined simple XML Schema datatypes

[*WEBARCH*] defines the following:

- content negotiation
- dereferences
- IRI collision
- IRI owner
- URI persistence

[*XML11*] defines the following:

- Char
- XML content

[*XMLSCHEMA11-2*] defines the following:

- doubleLexicalMap
- floatPtRound

xsd:anyURI

xsd:base64Binary

xsd:boolean

xsd:byte

xsd:date

xsd:dateTime

xsd:dateTimeStamp

xsd:dayTimeDuration

xsd:decimal

xsd:double

xsd:duration

xsd:ENTITIES

xsd:ENTITY

xsd:float

xsd:gDay

xsd:gMonth

xsd:gMonthDay

xsd:gYear

xsd:gYearMonth

xsd:hexBinary

xsd:ID

xsd:IDREF

xsd:IDREFS

xsd:int

xsd:integer

xsd:language

xsd:long

xsd:Name

xsd:NCName

xsd:negativeInteger

xsd:NMTOKEN

xsd:NMTOKENS

xsd:nonNegativeInteger

xsd:nonPositiveInteger

xsd:normalizedString

xsd:normalizedString

xsd:NOTATION

xsd:positiveInteger

xsd:QName

xsd:short

xsd:string

xsd:time

xsd:token

xsd:unsignedByte

xsd:unsignedInt

xsd:unsignedLong

xsd:unsignedShort

xsd:yearMonthDuration

J. References

J.1 Normative references

[BCP47]

Tags for Identifying Languages. A. Phillips, Ed.; M. Davis, Ed. IETF. September 2009. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc5646

[DOM]

DOM Standard. Anne van Kesteren. WHATWG. Living Standard. URL: https://dom.spec.whatwg.org/

[HTML5]

HTML5. Ian Hickson; Robin Berjon; Steve Faulkner; Travis Leithead; Erika Doyle Navara; Theresa O'Connor; Silvia Pfeiffer. W3C. 27 March 2018. W3C Recommendation. URL: https://www.w3.org/TR/html5/

[I18N-GLOSSARY]

Internationalization Glossary. Richard Ishida; Addison Phillips. W3C. 17 October 2024. W3C Working Group Note. URL: https://www.w3.org/TR/i18n-glossary/

[INFRA]

Infra Standard. Anne van Kesteren; Domenic Denicola. WHATWG. Living Standard. URL: https://infra.spec.whatwg.org/

[RDF11-TESTCASES]

RDF 1.1 Test Cases. Gregg Kellogg; Markus Lanthaler. W3C. 25 February 2014. W3C Working Group Note. URL: https://www.w3.org/TR/rdf11-testcases/

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC3629]

UTF-8, a transformation format of ISO 10646. F. Yergeau. IETF. November 2003. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3629

[RFC3986]

Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC3987]

Internationalized Resource Identifiers (IRIs). M. Duerst; M. Suignard. IETF. January 2005. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3987

[RFC7493]

The I-JSON Message Format. T. Bray, Ed. IETF. March 2015. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc7493

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[RFC8259]

The JavaScript Object Notation (JSON) Data Interchange Format. T. Bray, Ed. IETF. December 2017. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc8259

[RFC8615]

Well-Known Uniform Resource Identifiers (URIs). M. Nottingham. IETF. May 2019. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc8615

[Unicode]

The Unicode Standard. Unicode Consortium. URL: https://www.unicode.org/versions/latest/

[XML-NAMES]

Namespaces in XML 1.0 (Third Edition). Tim Bray; Dave Hollander; Andrew Layman; Richard Tobin; Henry Thompson et al. W3C. 8 December 2009. W3C Recommendation. URL: https://www.w3.org/TR/xml-names/

[XML11]

Extensible Markup Language (XML) 1.1 (Second Edition). Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau; John Cowan et al. W3C. 16 August 2006. W3C Recommendation. URL: https://www.w3.org/TR/xml11/

[XMLSCHEMA11-2]

W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes. David Peterson; Sandy Gao; Ashok Malhotra; Michael Sperberg-McQueen; Henry Thompson; Paul V. Biron et al. W3C. 5 April 2012. W3C Recommendation. URL: https://www.w3.org/TR/xmlschema11-2/

§ J.2 Informative references

[ABNF]

Augmented BNF for Syntax Specifications: ABNF. D. Crocker, Ed.; P. Overell. IETF. January 2008. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc5234

[COOLURIS]

Cool URIs for the Semantic Web. Leo Sauermann; Richard Cyganiak. W3C. 3 December 2008. W3C Working Group Note. URL: https://www.w3.org/TR/cooluris/

[did-core]

Decentralized Identifiers (DIDs) v1.0. Manu Sporny; Amy Guy; Markus Sabadello; Drummond Reed. W3C. 19 July 2022. W3C Recommendation. URL: https://www.w3.org/TR/did-core/

[HTML]

HTML Standard. Anne van Kesteren; Domenic Denicola; Dominic Farolino; Ian Hickson; Philip Jägenstedt; Simon Pieters. WHATWG. Living Standard. URL: https://html.spec.whatwg.org/multipage/

[HTML-RDFA]

HTML+RDFa 1.1 - Second Edition. Manu Sporny. W3C. 17 March 2015. W3C Recommendation. URL: https://www.w3.org/TR/html-rdfa/

[JSON-LD11]

JSON-LD 1.1. Gregg Kellogg; Pierre-Antoine Champin; Dave Longley. W3C. 16 July 2020. W3C Recommendation. URL: https://www.w3.org/TR/json-ld11/

[LINKED-DATA]

Linked Data Design Issues. Tim Berners-Lee. W3C. 27 July 2006. W3C-Internal Document. URL: https://www.w3.org/DesignIssues/LinkedData.html

[OWL2-OVERVIEW]

OWL 2 Web Ontology Language Document Overview (Second Edition). W3C OWL Working Group. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/owl2-overview/

[OWL2-SYNTAX]

OWL 2 Web Ontology Language Structural Specification and Functional-Style Syntax (Second Edition). Boris Motik; Peter Patel-Schneider; Bijan Parsia. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/owl2-syntax/

[RDF-CANON]

RDF Dataset Canonicalization. Gregg Kellogg; Dave Longley; Dan Yamamoto. W3C. 21 May 2024. W3C Recommendation. URL: https://www.w3.org/TR/rdf-canon/

[RDF-CONCEPTS-20040210]

Resource Description Framework (RDF): Concepts and Abstract Syntax. Graham Klyne; Jeremy Carroll. W3C. 10 February 2004. W3C Recommendation. URL: https://www.w3.org/TR/2004/REC-rdf-concepts-20040210/

[RDF11-CONCEPTS]

RDF 1.1 Concepts and Abstract Syntax. Richard Cyganiak; David Wood; Markus Lanthaler. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/rdf11-concepts/

[RDF11-DATASETS]

RDF 1.1: On Semantics of RDF Datasets. Antoine Zimmermann. W3C. 25 February 2014. W3C Working Group Note. URL: https://www.w3.org/TR/rdf11-datasets/

[RDF11-MT]

RDF 1.1 Semantics. Patrick Hayes; Peter Patel-Schneider. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/rdf11-mt/

[RDF12-INTEROP]

RDF 1.2 Interoperability. Pierre-Antoine Champin. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-interop/spec/

[RDF12-N-QUADS]

RDF 1.2 N-Quads. Gregg Kellogg; Dominik Tomaszuk. W3C. 22 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-quads/

[RDF12-N-TRIPLES]

RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 20 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RDF12-NEW]

What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]

RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SCHEMA]

RDF 1.2 Schema. Dominik Tomaszuk. W3C. 26 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-schema/

[RDF12-SEMANTICS]

RDF 1.2 Semantics. Peter Patel-Schneider; Dörthe Arndt; Enrico Franconi. W3C. 23 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-semantics/

[RDF12-TRIG]

RDF 1.2 TriG. Gregg Kellogg; Dominik Tomaszuk. W3C. 13 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-trig/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 15 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RDF12-XML]

RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[RDFA-CORE]

RDFa Core 1.1 - Third Edition. Ben Adida; Mark Birbeck; Shane McCarron; Ivan Herman et al. W3C. 17 March 2015. W3C Recommendation. URL: https://www.w3.org/TR/rdfa-core/

[RFC3023]

XML Media Types. M. Murata; S. St. Laurent; D. Kohn. IETF. January 2001. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3023

[RFC5890]

Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework. J. Klensin. IETF. August 2010. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc5890

[RFC5892]

The Unicode Code Points and Internationalized Domain Names for Applications (IDNA). P. Faltstrom, Ed. IETF. August 2010. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc5892

[RFC6874]

Representing IPv6 Zone Identifiers in Address Literals and Uniform Resource Identifiers. B. Carpenter; S. Cheshire; R. Hinden. IETF. February 2013. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc6874

[rfc7230]

Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing. R. Fielding, Ed.; J. Reschke, Ed. IETF. June 2014. Proposed Standard. URL: https://httpwg.org/specs/rfc7230.html

[SPARQL11-QUERY]

SPARQL 1.1 Query Language. Steven Harris; Andy Seaborne. W3C. 21 March 2013. W3C Recommendation. URL: https://www.w3.org/TR/sparql11-query/

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 26 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 23 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[STRING-META]

Strings on the Web: Language and Direction Metadata. Richard Ishida; Addison Phillips. W3C. 17 October 2024. W3C Working Group Note. URL: https://www.w3.org/TR/string-meta/

[SWBP-N-ARYRELATIONS]

Defining N-ary Relations on the Semantic Web. Natasha Noy; Alan Rector. W3C. 12 April 2006. W3C Working Group Note. URL: https://www.w3.org/TR/swbp-n-aryRelations/

[SWBP-XSCH-DATATYPES]

XML Schema Datatypes in RDF and OWL. Jeremy Carroll; Jeff Pan. W3C. 14 March 2006. W3C Working Group Note. URL: https://www.w3.org/TR/swbp-xsch-datatypes/

[UNICODE-SECURITY]

Unicode Security Considerations. Mark Davis; Michel Suignard. Unicode Consortium. 19 September 2014. Unicode Technical Report #36. URL: https://www.unicode.org/reports/tr36/tr36-15.html

[URL]

URL Standard. Anne van Kesteren. WHATWG. Living Standard. URL: https://url.spec.whatwg.org/

[VOCAB-ORG]

The Organization Ontology. Dave Reynolds. W3C. 16 January 2014. W3C Recommendation. URL: https://www.w3.org/TR/vocab-org/

[WEBARCH]

Architecture of the World Wide Web, Volume One. Ian Jacobs; Norman Walsh. W3C. 15 December 2004. W3C Recommendation. URL: https://www.w3.org/TR/webarch/

